

Final Report, MCS Internship

Name: Richard Blazek

Student Number: 22337668

Degree: Master in Computer Science

Company Name: Intel Ireland

Supervisor: Gareth Young

29th June 2026

Declaration

I hereby declare that this project is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

Richard Blazek

22337668

29th June 2026

Internship Goals

To make any meaningful contribution to the Intel NPU compiler, I first need to learn their software engineering practices and the compiler architecture, so these two goals have the highest priority. My first tasks will focus on refactoring the code, which gives me some time to learn how to find my way around the codebase. Finally, I can continue to the more technically interesting problems of improving compiler performance.

Learn the software engineering practices at Intel

This is my first time having a full-time job at a large company. Software development at large companies is more structured and requires more steps between writing the code and releasing the product to the customers. Compared to the environment at smaller companies or universities, there are more processes and rules that have to be followed. I need to learn the software practices at Intel and get used to following them so that I can be useful as a member of the team.

- **Specific:** I follow our coding standards and formatting guidelines. I can open pull requests, ask for code reviews, and maintainers merge my changes into the upstream. I know how to execute tests and write my own tests to check that my code functions as intended. I track my progress on tasks using Jira tickets.
- **Measurable:** I have had at least two pull requests integrated upstream. I have run CI (continuous integration) checks. I have closed a Jira ticket.
- **Achievable:** All the instructions are described in our documentation. If something is not clear to me, I can ask any of my colleagues to help me.
- **Relevant:** I need to learn and follow the practices of the company I work at.
- **Time-bound:** In the first two months of the internship, the company software engineering practices should become a habit.

Understand the Intel NPU compiler architecture

The Intel NPU compiler is a multi-pass compiler built using the MLIR framework. The MLIR stands for Multi-Level Intermediate Representation and it is a framework built on top of LLVM that allows developers to work with different dialects of the IR (intermediate representation) to represent the model at different layers, as the compilation goes from a machine learning model to an ELF binary. As a new intern on the compiler team, I need to understand this compiler architecture to know what I am even doing.

- **Specific:** I understand the MLIR framework and the IR dialects we use in different layers of our compiler. I am familiar with some of the passes and optimisations in our compiler.
- **Measurable:** I can explain the NPU compilation pipeline from a machine learning model down to an ELF binary. I can explain the different MLIR dialects used at different layers of compilation.
- **Achievable:** My team can explain the architecture to me and we also have documentation. Besides that, I have experience working with unfamiliar codebases in the past.
- **Relevant:** I need to understand the design of the system I am working with.
- **Time-bound:** I should be able to understand this after the first two months, although I will likely keep learning new things about the NPU compiler throughout the entire internship.

Improve code quality

My first assignments during the internship are focused on refactoring, such as moving optimisation logic from one pass into another and redesigning our compilation options. That will help me get familiar with the codebase before moving on to larger projects.

- **Specific:** I will complete my assignments related to improving code quality, including fixing any bugs encountered. I will write tests to check the modified code.

- **Measurable:** I have completed at least two refactoring Jira tickets and had them merged into the upstream.
- **Achievable:** These refactoring tickets are not technically difficult. If something is not entirely clear, I can ask my mentor or other team members.
- **Relevant:** Improving code quality makes maintenance easier and it helps me get familiar with the codebase.
- **Time-bound:** By the time of the midpoint report, these tickets should be done.

Improve compiler performance

One issue with the Intel NPU compiler is that it takes too long to compile models. The compiler team is working on ways to make compilation faster, preferably without sacrificing the performance of the compiled models. I will work on some of the related tasks, the first of which is a ticket on developing an analyser for detecting which compiler passes are unused and waste compilation time.

- **Specific:** I will complete the analyser for detecting unused compiler passes and add it to our CI to have it scheduled automatically. I will work on other refactoring tickets my mentor assigns to me.
- **Measurable:** I have developed the analysis tool for detecting unused compiler passes. Furthermore, I have merged a performance improvement pull request resulting in a quantifiable percentage reduction in the compilation time for the testing models.
- **Achievable:** Earlier tasks give me an understanding of the compiler. After completing them, I should be able to advance to these more challenging problems. And I can still rely on existing documentation and help from my mentor and team members.
- **Relevant:** Improving performance makes the compiler more useful for Intel's customers and it is an interesting technical problem for me.
- **Time-bound:** I will start after completing my refactoring tickets. By the end of the internship, I will have achieved measurable improvements in compilation time.

Reflective Diary

12 to 16 January

On Monday, all of us interns spent the day attending various orientation sessions. The instructors presented on health, safety, company policies and intellectual property. For the rest of the week, we were occupied with onboarding tasks such as obtaining the Intel employee badge, meeting my manager and team colleagues, and setting up my laptop and virtual machine. Before I could start working on the code, I was required to request several permissions in our access control system and wait to receive them.

By Friday, I had finished my onboarding and started my first assignment. It was a simple Jira ticket assigned to me by my mentor, which only required deleting one old unused class, submitting a pull request and getting it merged into develop (the main branch in our repository). I opened my first pull request and finished my first week at Intel.

19 to 23 January

I found out that the pull request I had opened was not yet ready, and that more steps were required between writing the code and integrating it upstream. I needed to learn how to run CI (continuous integration) tests to verify that my code could be safely merged and how to use rebasing to keep the Git history clean. I had to ask my mentor and another colleague to review and approve my changes. By the end of the week, my first ticket was done, and my changes were ready to be automatically merged.

Then my mentor described my second task, which involved refactoring the clamp-fusing logic in the NPU compiler. Clamp fusing means merging the clamping operation with the previous operation to improve the performance of the generated code. My job was to move this logic from a lower level of the compiler to a more abstract layer.

After work on Thursday, I had dinner with my colleagues from the compiler team, including some who were visiting from Romania.

26 to 30 January

I began the task I was assigned last week. When I ran the tests, I encountered an issue with old clamp-fusing tests that suddenly stopped working. I did not understand the problem, so I reverted my changes and tried to reimplement the logic. This time, I was very careful not to touch the existing code that the old tests covered, just adding my new code as a parallel implementation. As a result, I ended up with two parallel implementations of clamp fusing, the original one and my new one.

To write the tests for my implementation, I looked at the old clamp-fusing tests. There I noticed that some of the test cases did not make any sense, yet the tests always passed. I asked my mentor about that, and he checked the Git history of the tests. This way, we discovered that the tests were correct when created, but that later some of the test cases had been changed in a way that looked clearly wrong. Apparently, someone broke the code and decided to change the failing tests instead of fixing their code. This episode cost me two days, but it taught me why we ought to do test-driven development instead of retrofitting tests to existing code.

On Friday, I opened the pull requests for my ticket and started the CI pipeline.

2 to 6 February

Over the weekend, the CI finished, and some of the checks failed. I spent the week trying to fix the issues, which progressed slowly because running the full pipeline took around 24 hours, depending on how many other checks were in the queue. Therefore, any time I tried to fix something, it took an

entire day to see if the error was solved. Also, my mentor was away this week, so I was not always sure what to do.

I resolved some of the errors that were caused by bugs in my code. However, some tests in the CI behaved non-deterministically — even if there was no issue in the code, they failed with some non-zero probability. I requested additional permissions, which allowed me to restart a failed check, because some of them simply succeeded on a second or third try.

9 to 13 February

Another of my pull requests was merged, but my task required merging two more pull requests. On top of that, I received a new task to work on, refactoring the mechanism for disabling compiler passes. The compiler had too many command-line options for disabling various compiler passes, and different conditionals scattered around the codebase, each responsible for checking one option. My job was to replace this complexity with a single option holding the list of disabled passes and a single class responsible for executing passes, which checks whether a pass should be disabled before running it.

My manager gave me an overview of the design of the Intel NPU (Neural Processing Unit) and the compiler, and arranged one-on-one meetings with the senior programmers, so that each could introduce me to their part and I could understand the whole architecture.

16 to 20 February

I attended two individual meetings with senior programmers explaining the compiler structure to me. The first one covered the lower layers of the compiler and some optimization techniques she was implementing. The second one (my mentor) walked me through the more abstract layers of the compiler and described the design choices.

I succeeded in getting a second pull request from my first task merged, and I was simultaneously working on my second task (because I was free while waiting for CI checks on the first task). I created the command-line option and the class with a unit test. Then I started the excruciating process of crawling through the codebase in search of options to remove. Did I mention the options were scattered all around? Another issue was that it was not even clear which options should be removed and which ones were actually necessary, and the program would break without them.

23 to 27 February

Another senior programmer explained his part of the compiler, the scheduler, to me.

I discussed my second ticket (compiler pass disabling) with my mentor and another programmer. We agreed I should split the ticket into three smaller pull requests rather than implementing it all at once, so that they could review each pull request. I learnt the difference between an epic and a story in Scrum. The pass-disabling ticket was an epic, but each pull request had to be linked to a story, so I created three Jira stories for the three pull requests I had to make.

The first story was implementing a utility class responsible for conditional pass execution, checking whether a pass is disabled and executing it only if it is not. The second one was adding the pass-disabling option and using the utility class to execute only passes not listed in that option. The last story required removing all the legacy options.

2 to 6 March

I completed the first story for the pass-disabling epic, created a pull request and had two reviewers approve and merge it. I also created one more pull request for my previous story (clamp fusing) because some old code was now obsolete and had to be removed. Hopefully it is the last pull request for this ticket, so I can be done with it soon.

9 to 13 March

This week, I implemented the second story of the pass-disabling epic and got it merged. In order to finally mark this epic as closed, I needed to create one last pull request to remove the old pass-disabling options. I wrote a Python script that scanned the source tree and collected the options that were always enabled by default, so that I wouldn't need to look for them manually. It found around fifty of them, but some were false positives because they were not related to pass disabling. So I still had to check, for each option, whether it should be removed, and adjust the code that used it.

Of course, I discovered that I needed to make another change to the code before we could close the clamp-fusing story. Thus, I created yet another pull request. This story is a never-ending story. On the other hand, next week I should finish both tasks and progress to something completely different.

16 to 20 March

At last, I merged the last pull request for the clamp-fusing ticket, so I could close the ticket in Jira and turn to the next task. Pass disabling still required some more changes from code reviewers and did not pass the automated checks, so I did not get it merged by Friday.

Tuesday of this week was St Patrick's Day, which was a bank holiday, and many colleagues took a day off on Monday to extend their weekend. On Wednesday, I was surprised by how busy the office was and how mentally alert my colleagues were the morning after St Patrick's Day.

Back in the office, my mentor explained my next ticket to me, which focused on optimizing the compiler rather than refactoring. The goal was to write an analysis tool that could detect which compiler passes were outdated and no longer modified the compiled model.

23 to 27 March

When the last pass-disabling ticket was being merged, some merge conflicts occurred and caused the merge to fail. I had to find out which parts caused the errors and fix them.

Meanwhile, I implemented the code for detecting the unused passes. It would calculate the hash of the compiled model before and after each pass, compare these to see whether there had been any change, and write the results into a log file. I added a compiler option that allowed enabling this analysis and specifying the log file path.

30 March to 3 April

To test my unused pass detection tool, I needed to download the testing models, compile them with my analysis enabled and then collect the logs. For this, I used a Python helper that the team was already using for testing. I adapted the helper for my purpose and started downloading and compiling all models that were stored in our cloud. There were around 11,000 models, so this process took a long time, and I kept it running overnight.

The next day, my mentor asked me about it, and I explained that I needed to wait for the 11,000 models to compile. Then he explained to me that we were definitely not going to wait for 11,000 models to compile, and that I should ask a colleague responsible for the CI which models were important for testing. I asked him, and he told me that he would prepare the list and send it to me.

6 to 10 April

The colleague who promised to send me the list of models used for testing did not send me anything, so I reminded him, and he told me it was in progress. I was starting to think that if I hadn't bothered asking him and had simply run the analysis on all models instead, it would have been finished by then. In the meantime, I wrote documentation for the new pass-disabling mechanism and merged it into the repository.

On Thursday, my manager asked me how my internship was going and whether I would consider working at Intel after graduation. I replied that I thought my internship was going well and that I would be happy to work there.

13 to 17 April

My mentor introduced me to another problem the team was working on, which was compiling repeating blocks. Many machine learning models (such as LLaMA) consist of blocks of several operations repeated many times (e.g. Convolution, Clamp, BatchNorm). At the time, the compiler had to do all compilation steps for each repetition separately, which could take a long time for large models. If we compiled such a repeating block only once and then called the compiled block repeatedly, the compilation could be much faster.

The first ticket pertaining to this problem was adding support for repeating blocks in a compiler pass for serializing ELF files (represented as IR) into binary data. Previously, the pass would crash if its argument was a repeating block, and my goal was to have it generate the correct output instead.

My mentor had a working version of that pass in another branch, which, however, contained unrelated changes and was not considered “pretty”. Nevertheless, I could compile a sample model (with repeating blocks) in that branch and dump the IR before and after the pass. This way, I obtained the reference input and output for the pass, from which I created a test. Now I only had to modify the pass to pass (pun not intended) the test.

20 to 24 April

I made the changes required to get the pass working correctly. My mentor also told me to refactor the tests for the pass because they defined many attributes that were not relevant for testing the pass. After that, my pull request got merged and the ticket was closed.

My manager asked me about my unused pass detection tool, so I informed him that I still hadn’t received the list of the CI testing models. He scheduled an online call with me and one person responsible for the CI, who explained to me how to connect to our AWS S3 bucket and which models to download from it. However, that required me to first ask for permissions to access the bucket.

There was one simple ticket, which required changing the default settings for repeated-block compilation. The change was passing weights as arguments to repeated blocks, rather than embedding them as constants inside the block. The original behaviour produced incorrect programs when repetitions used different weights, because the weights from the first repetition were embedded in the block. Closing this story required nothing more than toggling the default value of a flag in the settings and updating the tests to reflect the new behaviour.

Another ticket I started was lazy evaluation for one expensive operation that was slowing down our tests. Every test was evaluating that operation even though most tests did not need its result. Lazy evaluation ensured we did not evaluate it unless we needed it, making our test suite run three times faster.

27 April to 1 May

Both of my pull requests from last week were accepted into the main repository. Then I finally tested my unused pass detection tool on around 500 models for the Intel Panther Lake architecture. When I saw the results, I couldn’t believe my eyes. I sent my results to my mentor, who posted them in a Microsoft Teams group chat for compiler developers, and they couldn’t believe their eyes either. Surely, it was not possible that 92 out of 484 passes were useless?

Having such groundbreaking results at least gave me something to talk about during the internship presentation for our Trinity supervisor on Friday of this week. Our supervisor told me and my fellow intern Ayushmaan that it looked like our internship was going well.

4 to 8 May

This week I began integrating the unused pass detection tool into the compiler and merging it into develop so that it could be run automatically in the CI.

Besides that, I returned to repeating blocks. Our compiler has a pass for detecting repeating blocks. After identifying them, it extracts them into subroutines, which can be reused, to make the model smaller and faster to compile. The result is a main program that runs on the CPU and is responsible for calling the subroutines on the NPU.

The problem is that the CPU should only call the subroutines, and not execute NPU instructions. Because of that, the compiler should not leave any NPU instructions in the main program; it should extract them all into subroutines. The pass used to extract only repeating blocks, so my job was to add extra code to find leftover NPU instructions and put them into separate subroutines.

11 to 15 May

I modified the pass that extracts repeating blocks into subroutines for the task I started last week. I added some extra code that went through the main program, collected the remaining NPU instructions and created subroutines for them, so that no NPU-related operations were left in the main program. My mentor told me that we should make the code more generic by moving this new code into a separate class before we closed this task.

I implemented the integration of the unused pass detection tool and ran the analysis manually in the CI to get results for more models than the 500 I had used for testing before, and for architectures other than just Panther Lake. Still, the result was that 62 passes were never used in any of these cases, and 180 passes (about a third) were used less than one per cent of the time.

18 to 22 May

I wanted to merge my pull request to integrate the unused pass detection. However, one of the senior programmers doing code review said it conflicted with another programmer's work. We scheduled a call to figure it out. The issue was that there was an MLIR instrumentation to set a callback that is run whenever a pass is executed, which both the other programmer and I used. Since there could only be one such callback, we couldn't both merge our pull requests without conflict.

Fortunately, there was a class in MLIR for this purpose that allowed registering multiple observers to be called before and after a pass is executed. However, this class was not memory-safe, worked with plain pointers and relied on its user to keep track of object lifecycles. Therefore, the first step was implementing a wrapper using RAII to manage memory automatically, which I promptly did, and my pull request was approved without any delay. The next required change was modifying the old pass-disabling logic I had implemented earlier to work with this new class.

25 to 29 May

I modified the pass-disabling logic to work with the new class I created last week. I created the pull request, waited for it to be approved and merged, and then I finally went back to my original pull request integrating the unused pass detection into the compiler. All I needed to do was adapt my class for monitoring pass usage so that it could be registered as an observer that is called before and after a pass. Then I asked for code review again.

In parallel, I kept working on collecting leftover NPU instructions and extracting them into subroutines. As my mentor had previously requested that I make my code more generic, I refactored the existing code for extracting repeating blocks into subroutines to simplify and generalize it. I opened and merged the refactoring pull request, but I still had to merge the pull request that was actually adding the new logic to collect leftover NPU instructions.

1 to 5 June

In the code review on my unused pass detection PR, I got two approving reviews, meaning I could have it merged upstream into the compiler. Likewise, my pull request extracting leftover NPU instructions into subroutines was approved and merged.

Next issue I was asked to investigate was one model with repeated blocks crashing during compilation. I was debugging the issue but every time I fixed the crash in one pass, the compilation would crash in a later pass. Eventually, I had to deal with another tasks, not having resolved this problem.

8 to 12 June

This week I started working on a new task which was implementing a pass for extracting constant transformations. It would collect operations which performed transformations of constant model weights and apply these transformations on those weights at compile time. The goal was that these computations would not have to run during model inference time, improving the performance at inference time.

15 to 19 June

I had a call with my mentor where we discussed the last week's ticket. We discussed how the pass should decide certain edge case scenarios and I defined some tests to describe the behaviour for the pass.

Independently, I decided to try deleting one of the passes that our analysis previously deemed unused and open a pull request with that change. The pull request with pass deletion got two approving reviews and was subsequently merged. Thus, my unused pass detection was slowly but surely starting to have some impact.

22 to 26 June

My manager asked me to delete multiple unused passes in bulk (say 20) so that we can see if there are any errors reported by the CI, or if it is perfectly all right to start eliminating passes en masse.

I also opened a pull request with the pass for extracting constant transformations. From Thursday, I was away on a family vacation.

29 June to 3 July

On Monday and Tuesday, I was still away, coming back on Wednesday. I had a meeting with my mentor and my manager, both of which agreed that I was doing good job and they would like to hire me at Intel after graduation.

I made some further changes to the pass for extracting constant transformations and asked for code review. The other pull request, which deleted multiple passes, failed because some of the passes were in fact necessary on some platforms. The problem was that I previously had not run the analysis on all platforms, so some of the unused passes were in fact used on some platforms I did not test.

6 to 8 July

I was finishing this week, because I was taking days off for the next week and for the Thursday and Friday of this week. Therefore, I had to make my work-in-progress ready for my mentor to take over.

I did not make further progress to the pass for extracting constant transformations, as that was not a priority and I only had last three days. I fixed one error that previously caused the compilation of one model with repeating blocks to crash and submitted a PR; the compilation would still crash but at a later stage which was considered an improvement.

For the pass usage analysis, my mentor ran it in the CI for all models on all platforms, so I had to write a Python script to extract the results from the CI output. This meant that by the end of this week, they could complete the report with all the data and start deleting the passes. I hope that by the time I return to Intel next year, these passes will be gone.

Technology Management Processes

Our team

I am the only intern on the Intel NPU compiler team in Leixlip. My manager does not direct the daily work I am doing; I work with my mentor and occasionally other senior engineers. Our team members mostly work from our Leixlip office in Ireland, though there is another team in Timisoara, Romania, working on the compiler. For some pull requests, I need to ask some of the Romanian software engineers for a code review.

Communication

For sharing information about specific tasks in a structured manner, our team relies on GitHub pull requests and Jira tickets, which are related to a specific task. In addition, we hold regular weekly meetings on Thursdays, where we discuss progress on tasks during the past week and plans for the next week.

For instant communication, we rely on Microsoft Teams. Given that most employees come to the office on some workdays, it is often possible to discuss things in person.

Quality assurance

Software development at the Intel NPU compiler team is very rigorous. There are multiple steps between writing the code and releasing the product, to verify that everything works as it should. Every pull request requires approval from two code reviewers and must pass the CI (continuous integration) checks. After that, a maintainer adds an auto-merge label to the pull request and an automated Jenkins job merges the changes into the upstream at an appropriate time.

This automated merging is necessary because the CI pipeline has to be executed again after every change to the upstream to pre-empt the possibility that two changes from two different pull requests interact with each other in unpredictable ways.

Project management methods

The team follows the Agile and Scrum methodologies to manage the software development lifecycle. Large tasks are categorised as *epics*, which are then broken down into smaller *stories*. Jira is the primary tool used to track progress on tasks, stories and epics. This ensures that progress on tasks can be tracked and each pull request is linked to a specific task.

Software development tools

We use several software development tools to enable our workflow:

- Git: Git is used for version control; rebasing is preferred over merging to keep the Git log clean
- GitHub: GitHub stores our repository online, each developer has their own fork and submits pull requests to have their work merged into the upstream
- Jenkins: Jenkins runs the CI checks and merges pull requests into the upstream
- Automation: Python scripts and helper tools are used to automate repetitive tasks, such as downloading testing models or text processing
- Copilot: GitHub Copilot assists during development. The codebase of the compiler is too large and complex for Copilot to be reliable at code generation, but it is useful for debugging and for explaining the code.

Critical evaluation

Strengths

- The strict code review process ensures that code changes are independently verified by multiple engineers before they are accepted.
- The CI pipeline thoroughly tests all pull requests to catch possible errors and performance regressions in the compiler.

Weaknesses

- The CI pipeline is very slow, with checks taking around 24 hours to complete, which is a significant bottleneck in the development process.
- Some of the CI checks are flaky and fail even for correct code. They have to be restarted manually to pass on a second or third try, wasting developers' time.